



RAG Pipeline

AICB-P1 · Ngày 8 · Retrieval — Augmentation — Generation

M.Sc Trần Minh Tú

Senior AI Engineer @ Obello

Squad Leader – AI Automation & Knowledge Base @ MSB

VinUniversity · Phase 1 · Tuần 2 · 2026

HÃY SUY NGHĨ...

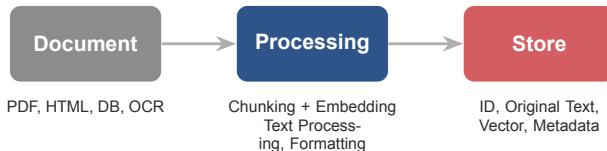
*“Bạn đã build agent với vector store.
Nhưng agent vẫn hallucinate và trả lời
sai. Lỗi nằm ở đâu trong pipeline?”*

Giữ câu hỏi này trong đầu khi học bài hôm nay

Retro Day 07

Nhìn lại Indexing Pipeline đã xây hôm qua, xác định điểm cần nâng cấp, và kết nối sang Retrieval Pipeline

01



Mỗi record trong store cần tối thiểu: **ID · Original text · Vector · Metadata** (source, section, date, access).



Lưu ý: Pipeline Day 07 chỉ có dense search + threshold + top-k + inject thô vào LLM. Chưa có hybrid, rerank, augmentation, hay evaluation.

Ingestion (Offline)

- Document → Processing → Store
- Chạy batch hoặc near real-time
- Mục tiêu: chuẩn bị dữ liệu sạch, có metadata, sẵn sàng search

Retrieval (Runtime)

- Query → Search → Filter → LLM → Answer
- Chạy mỗi khi user hỏi
- Mục tiêu: tìm đúng chứng cứ, sinh câu trả lời grounded

Ingestion tạo ra index. Retrieval dùng index đó để trả lời. Chất lượng ingestion quyết định trần của retrieval.

Day 07: Retrieval Pipeline

- Dense search only
- Inject context thô vào prompt
- Không có rerank, không có evaluation
- “Có câu trả lời” nhưng chưa biết đúng hay sai

Day 08: RAG Pipeline

- Hybrid search + rerank
- Augmentation: đóng gói context có cấu trúc
- Generation: grounded prompt + citation
- Evaluation: đo faithfulness, relevance, recall

Lưu ý: Day 08 nâng cấp retrieval pipeline thành **RAG pipeline** hoàn chỉnh: R (search + rerank) → A (đóng gói context) → G (sinh câu trả lời grounded).

RAG — Retrieval-Augmented Generation

RAG không phải chỉ là gắn thêm context; nó là sự phối hợp giữa retrieval system, augmentation layer, và generation system

Kiến thức bị đóng băng

Knowledge Cutoff

LLM chỉ biết những gì đã xảy ra **trước ngày training**. Thông tin nội bộ hay sự kiện mới là **điểm mù**.

Bản chất xác suất

Probabilistic Nature

LLM là cỗ máy dự đoán từ tiếp theo, ưu tiên sự **trôi chảy** (fluency) hơn tính **chính xác** (factual accuracy).

Hệ quả — Hallucination

Khi thiếu dữ kiện, model sẽ tự động **“bịa”** ra thông tin trông rất logic và tự tin để **làm hài lòng** người dùng.

Lưu ý: Có thể còn nhiều lý do khác nữa và mấu chốt rằng, LLM không biết bản thân không biết. Đây là lý do chính để cần một hệ thống **cung cấp chứng cứ thực** trước khi sinh câu trả lời.

RAG — Retrieval-Augmented Generation — Kỹ thuật kết hợp **truy xuất thông tin** (retrieval) với **sinh ngôn ngữ** (generation) để LLM trả lời dựa trên dữ liệu thực thay vì chỉ dựa vào bộ nhớ huấn luyện.

R — Retrieval

Truy xuất thông tin từ kho dữ liệu ngoài (vector store, DB, search engine).

▷ Tìm đúng chứng cứ, lọc nhiễu, xếp hạng relevance.

A — Augmentation

Tăng cường prompt: đóng gói context có cấu trúc, gắn metadata, tách evidence vs question.

▷ Giảm noise, chống “lost in the middle”.

G — Generation

Sinh câu trả lời bám sát chứng cứ đã augment, kèm trích dẫn nguồn.

▷ Grounded answer, citation, self-check.

RAG = tìm chứng cứ → đóng gói có cấu trúc → sinh câu trả lời grounded. Kết quả có thể trích dẫn và kiểm chứng.

R — Retrieval

- Dense Search (Semantic)
- Sparse Search (BM25)
- Hybrid Search
- Rank Fusion (RRF)
- Reranking (Cross-encoder)
- MMR (đa dạng kết quả)
- Pre-filtering (Metadata)
- Query Routing
- Multi-index Search
- Parent-child Retrieval

A — Augmentation

- Context Injection
- Document Reordering
- Instruction Tuning
- Metadata Integration
- Citation Formatting
- Context Compression
- Token Budget Management
- Conflict Resolution
- Grounding Constraints
- Context Deduplication

G — Generation

- Grounded Generation
- Self-correction / Self-check
- Output Formatting
- Citation Generation
- Abstention (từ chối)
- Chain-of-Thought (CoT)
- LLM Selection / Routing
- Safety & PII Filtering
- Streaming Generation
- Multi-step Generation

Lưu ý: Mỗi kỹ thuật sẽ được trình bày chi tiết trong các phần R, A, G tiếp theo. Slide này là **bản đồ tổng quan** để định hướng.

Cần RAG khi

- Dữ liệu nội bộ thay đổi liên tục (policy, SOP, ticket)
- Cần câu trả lời có nguồn, kiểm chứng được
- Fine-tuning quá đắt hoặc dữ liệu quá nhạy cảm
- Cần giảm hallucination một cách có hệ thống

Chưa cần RAG khi

- Câu hỏi thuộc kiến thức chung (LLM đã biết)
- Không có corpus riêng để truy xuất
- Task không cần citation (creative writing, brainstorm)
- Prompt engineering đơn giản đã đủ

Dữ liệu nội bộ = Tài sản

Policy, SOP, hợp đồng, ticket, báo cáo — LLM **không hề biết** những thứ này.

Fine-tuning để nhồi facts rất đắt và dễ bị **catas-trophic forgetting**.

RAG = “open-book exam” — mang tài liệu vào phòng thi thay vì học thuộc lòng.

Compliance & Trách nhiệm

Ngành tài chính, y tế, pháp lý — câu trả lời **phải có nguồn** và **kiểm chứng được**.

Hallucination trong doanh nghiệp = **rủi ro pháp lý**, mất uy tín, thiệt hại tài chính.

RAG cho phép **trích dẫn** và **audit trail** cho mọi câu trả lời.

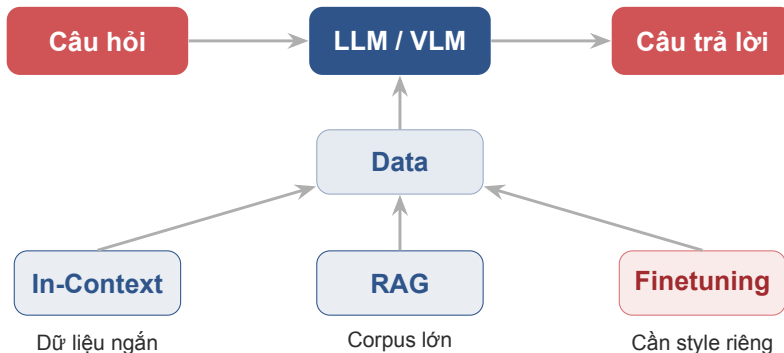
Cập nhật liên tục

Dữ liệu doanh nghiệp thay đổi **hàng ngày** (policy mới, ticket mới, sản phẩm mới).

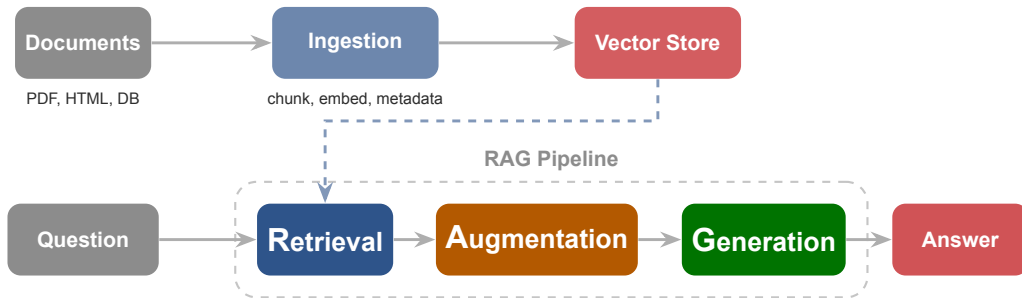
Fine-tuning lại model mỗi ngày? **Không khả thi**.

RAG chỉ cần cập nhật **index** — model không cần retrain.

Lưu ý: RAG không phải lựa chọn — nó là **yêu cầu bắt buộc** khi doanh nghiệp cần AI trả lời dựa trên dữ liệu nội bộ, có trách nhiệm, và luôn cập nhật.



In-Context: nhét thẳng vào prompt (ít dữ liệu, nhanh). **RAG:** tìm từ corpus lớn rồi inject (nhiều dữ liệu, cập nhật liên tục). **Finetuning:** huấn luyện lại model (cần style/domain riêng, đắt hơn).



Offline: Document → Ingestion → Vector Store (đã xây ở Day 07).

Runtime: Question → **Retrieval** → **Augmentation** → **Generation** → Answer.

R — Retrieval

Tìm đúng chứng cứ là bước quan trọng nhất; search sai thì prompt đẹp đến đâu cũng khó cứu

03

Dense Vector (Semantic)

Vector **ngắn, đặc** (768–1536 chiều), mọi chiều đều có giá trị $\neq 0$.

Tại sao gọi Dense? Vì embedding model nén **toàn bộ ngữ nghĩa** vào một vector dày đặc thông tin.

- + Hiểu nghĩa, paraphrase, cross-lingual
- + “hoàn tiền” $\approx$ “refund”
- Bỏ lỡ keyword chính xác (mã lỗi, tên riêng)
- Phụ thuộc chất lượng embedding model

Sparse Vector (BM25 / TF-IDF)

Vector **rất dài** (= kích thước từ điển) nhưng hầu hết chiều = 0.

Tại sao gọi Sparse? Vì chỉ các từ xuất hiện trong document mới có giá trị, phần còn lại là **số 0** (thừa thớt).

- + Match chính xác từ khóa, mã lỗi, tên riêng
- + Nhanh, không cần GPU
- Không hiểu đồng nghĩa, paraphrase
- “hoàn tiền” $\neq$ “refund”

Lưu ý: Không bên nào hoàn hảo. Dense mạnh về **ngữ nghĩa**, Sparse mạnh về **từ khóa chính xác**. Giải pháp: kết hợp cả hai $\rightarrow$ **Hybrid Search**.

Cách hoạt động

- Query và document đều được **embed** thành dense vector
- So sánh bằng **cosine similarity** hoặc dot product
- Hiểu được **nghĩa**, paraphrase, cross-lingual

Giới hạn

- Bỏ lỡ **keyword chính xác**: mã lỗi, tên riêng, số ticket
- Phụ thuộc vào chất lượng embedding model
- Không phân biệt được từ đồng âm khác nghĩa

Corpus nhiều câu tự nhiên, FAQ, knowledge base. Dense search là baseline tốt nhưng **không đủ cho production**.

Cách hoạt động

- Đếm tần suất từ (TF-IDF / BM25)
- Tạo **sparse vector**: hầu hết chiều = 0
- Match chính xác **từng từ**, không cần hiểu nghĩa

Giới hạn

- Bỏ lỡ **paraphrase** và đồng nghĩa
- “hoàn tiền” vs “refund” → không match
- Không hiểu ngữ cảnh, chỉ đếm từ

Policy, code, luật, log, mã ticket — nơi **exact term** quan trọng hơn nghĩa.

Hybrid = Dense + Sparse

- Chạy **cả hai** search song song
- Gộp kết quả bằng **RRF** (Reciprocal Rank Fusion) hoặc alpha weighting
- Giữ cả **nghĩa** lẫn **keyword**

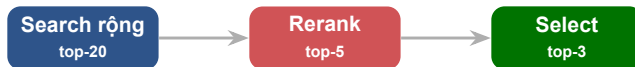
RRF Formula

$$\text{RRF}(d) = \sum_{r \in \text{rankers}} \frac{1}{k + \text{rank}_r(d)}$$

Tài liệu top cao ở **cả hai** bảng xếp hạng sẽ vươn lên vị trí 1.

Alpha tuning: $\alpha \cdot \text{Dense} + (1 - \alpha) \cdot \text{Sparse}$

Lưu ý: Hybrid search thường đáng thử đầu tiên khi corpus có cả ngôn ngữ tự nhiên lẫn tên riêng, mã lỗi, điều khoản.



Cross-encoder Reranker

Query + chunk ghép thành 1 input → relevance score.

Chính xác hơn bi-encoder nhưng chậm → chỉ dùng cho list nhỏ (top-20).

MMR

Giữ relevance nhưng giảm trùng lặp.

Chọn tập context **vừa đúng vừa đa dạng** trước khi đưa vào prompt.

Lưu ý: Mục tiêu retrieval không phải **lấy nhiều**, mà là lấy **đúng và đủ** cho augmentation.

A — Augmentation

Retrieval tìm chunk. Augmentation đóng gói chunk thành context có cấu trúc trước khi đưa vào LLM.

04

Cách sắp xếp tài liệu

- Tài liệu quan trọng nhất nên đặt ở **đầu** hay **cuối**?
- LLM nhớ tốt thông tin ở đầu và cuối prompt
- Thông tin ở **giữa** dễ bị “quên”

Lost in the Middle

- Chunk quan trọng vô tình nằm giữa → RAG thất bại
- **Giải pháp:** Document Reordering — đặt tốt nhất ở đầu, tốt thứ 2 ở cuối
- Thứ tự: [1, 3, 5, 4, 2]

Lưu ý: Context injection không phải chỉ nối chunk lại. Thứ tự, cấu trúc, và ranh giới giữa các nguồn đều ảnh hưởng đến chất lượng.

Viết chỉ dẫn rõ ràng để LLM biết:

- Đây là “sự thật khách quan” (Context / Evidence)
- Đây là câu hỏi của người dùng (Question)
- Đây là instruction cho model (System)

Dùng **XML tags**, **Markdown headers**, hoặc **delimiters** rõ ràng để tách các phần.

```
<system>
Answer only from the context below.
Cite [doc_id] when possible.
</system>

<context>
[1] policy/refund-v4.pdf | Section 3
Yeu cau hoan tien trong 7 ngay...
</context>

<question>
Chinh sach hoan tien la gi?
</question>
```


Strict Constraints

Ép LLM **chỉ được dùng** context cung cấp. Nếu thiếu chứng cứ → nói “không đủ dữ liệu” thay vì bịa.

Metadata Integration

Đưa thêm **thời gian, tác giả, phòng ban** vào context để LLM phân biệt được văn bản cũ/mới, ưu tiên nguồn mới nhất.

Citation Formatting

Yêu cầu LLM trả về **số thứ tự tài liệu** (ví dụ: [1], [2]) để người dùng có thể **kiểm chứng** (verify).

Grounding = “neo” LLM vào thực tế. Không có grounding, model sẽ tự điền khoảng trống bằng suy đoán.

Context Compression

- Dùng model nhỏ để **nén** các đoạn văn bản thô
- Chỉ giữ lại **ý chính** trước khi đưa vào LLM
- Giảm token, giảm noise, giảm “lost in the middle”
- Ví dụ: LLMLingua, Recomp

Token Budget

- Context chiếm gần hết cửa sổ → model khó ưu tiên đúng
- Giữ khoảng trống cho: instruction, question, output
- Rule of thumb: context $\leq 60\%$ token budget

Lưu ý: Augmentation checklist: (1) Pattern inject? (2) Token budget? (3) Evidence block có source? (4) Conflict xử lý? (5) Document ordering?

G — Generation

Augmentation đóng gói context. Generation sinh câu trả lời grounded từ context đã đóng gói.

05

Model lớn (GPT-4, Gemini Pro)

- Suy luận phức tạp, so sánh nhiều nguồn
- Câu hỏi cần tổng hợp, phân tích
- Chi phí cao, latency cao
- Phù hợp: internal tools, low-volume

Model nhỏ/local (Llama 3, Mistral)

- Câu hỏi đơn giản, lookup trực tiếp
- Dữ liệu nhạy cảm, cần on-premise
- Chi phí thấp, latency thấp
- Phù hợp: high-volume, data-sensitive

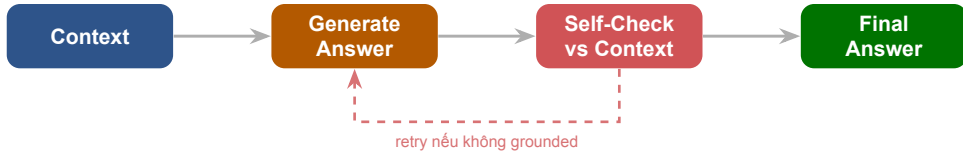
Chọn model dựa trên **độ phức tạp câu hỏi, yêu cầu bảo mật, và ngân sách**. Không phải lúc nào cũng cần model lớn nhất.

Formatting

- Yêu cầu output dạng **Markdown**, **Table**, hoặc **JSON**
- Phục vụ tích hợp vào hệ thống khác (API, dashboard)
- Dùng `response_format` hoặc instruction rõ ràng

Safety & Alignment

- Đảm bảo câu trả lời **không vi phạm chính sách**
- Không rò rỉ dữ liệu nhạy cảm không mong muốn
- Kiểm tra PII, access level trước khi trả về user



LLM tự đánh giá câu trả lời của chính mình so với context **trước khi** hiển thị cho người dùng. Nếu phát hiện suy diễn vượt chứng cứ → retry hoặc abstain.

Strict Grounding

Ép LLM **chỉ dùng context được cấp**, cấm dùng kiến thức từ training.

Nếu đổi context sai, model cũng phải trả lời sai theo context đó.

▷ Trọng tài duy nhất = dữ liệu nội bộ.

Forcing Citations

RAG mất 50% giá trị nếu model không chỉ ra **lấy từ tài liệu nào**.

Chỉ thị: “Trích dẫn [doc_id] khi đưa ra tuyên bố.”

▷ Output: “Nghỉ 14 ngày phép [doc_3].”

Graceful Degradation

Khi context không chứa đáp án → **từ chối** thay vì đoán mò.

Không chỉ nói “Không biết” cụt lùn — gợi ý bước tiếp theo cho user.

▷ “Chưa tìm thấy X. Bạn thử từ khóa Y hoặc liên hệ HR?”

Lưu ý: Grounding = “neo” LLM vào thực tế. Abstention = biết giới hạn. Cả hai là **tính năng quan trọng nhất** của RAG production.

Yêu cầu model “**suy nghĩ ra nháp**” trước khi đưa ra câu trả lời cuối:

- Lọc câu liên quan trong context
- Phân tích trong thẻ <thought_process>
- Tổng hợp thành câu trả lời

Câu hỏi so sánh, suy luận logic, tổng hợp từ nhiều nguồn. Không cần cho câu hỏi lookup đơn giản.

```
<system>
Lọc các câu liên quan trong context.
Phân tích trong <thought>.
Sau đó tổng hợp câu trả lời.
</system>

<thought_process>
Doc_1 nói ngày 12 ngày (2024)
Doc_3 nói ngày 14 ngày (2026)
→ Ưu tiên doc mới nhất
</thought_process>

Answer: Ngày 14 ngày phép [doc_3].
```


Xung đột ngữ cảnh

2 tài liệu mâu thuẫn (VD: 12 ngày vs 14 ngày phép).

LLM bối rối, chọn bừa hoặc cộng gộp.

Fix: “Ưu tiên tài liệu mới nhất, hoặc liệt kê cả 2 và chỉ ra mâu thuẫn.”

Suy diễn quá đà

Tài liệu: “Free ship đơn 500k ở HN”. User hỏi “HCM thì sao?”

LLM tự suy: “HN được thì HCM chắc cũng được” → hallucination.

Fix: “Không tự suy luận điều kiện không được đề cập rõ ràng.”

Bỏ qua constraints

Đã dẫn trích dẫn [doc_id] nhưng model quên. Hay gặp với model nhỏ hoặc context dài.

Fix: Đặt rule quan trọng ở **cuối prompt** (gần “Answer:” nhất). Giảm temperature = 0.

In formatted_context ra log. Nếu context **có** đáp án → lỗi Generation (sửa prompt/model). Nếu **không** có → lỗi Retrieval.

Pre RAG Technique

Query Transformation — kỹ thuật tiền xử lý câu hỏi trước khi đưa vào RAG pipeline

06



Lọc theo metadata

Department: chỉ tìm trong doc của phòng CS · **Date:** chỉ lấy policy còn hiệu lực · **Doc type:** chỉ SOP, bỏ qua FAQ · **Access level:** lọc theo quyền user

Index lớn → search chậm và nhiều noise. Pre-filter thu gọn scope **trước khi** chạy vector search, giúp **tăng tốc** và **tăng precision** cùng lúc.

Multi-Query

- Tự động tạo **nhiều biến thể** của câu hỏi
- Mỗi biến thể search riêng → gộp kết quả
- Tăng **recall**: bắt được chunk mà query gốc bỏ lỡ

“Chính sách hoàn tiền” → “refund policy” + “điều kiện trả hàng” + “thời hạn hoàn tiền”

HyDE

- **Hypothetical Document Embeddings**
- LLM sinh một **câu trả lời giả định** cho query
- Embed câu trả lời giả → search bằng vector đó
- Cải thiện khi query ngắn, mơ hồ

Query: “SLA P1?” → HyDE: “SLA ticket P1 là phản hồi 15 phút, xử lý 4 giờ...”

Lưu ý: Đừng thêm transformation chỉ vì framework có sẵn. Dùng khi query ngắn, mơ hồ hoặc có nhiều alias

Query Expansion

Chữa lỗi chính tả, thêm **từ đồng nghĩa** và thuật ngữ chuyên ngành.

▷ Tăng **Recall** — không bỏ sót tài liệu chỉ vì user dùng sai từ.

“hoàn tiền” → *“refund”, “trả hàng”, “charge-back”*

Query Decomposition

Tách câu hỏi phức tạp (multi-hop) thành **nhiều câu hỏi nhỏ**, search song song, rồi gộp context.

▷ Một vector không thể trả lời 2 ý khác biệt cùng lúc.

“So sánh hoàn tiền Shopee vs Tiki” → *Q1 + Q2*

Step-Back Prompting

Khi câu hỏi quá chi tiết, LLM sinh câu hỏi **“lùi một bước”** (abstract) để lấy ngữ cảnh chung trước.

▷ Tránh bị “lạc” trong tiểu tiết.

“Lỗi 404 API Momo user 8910” → *“Kiến trúc tích hợp Momo?”*

Pre-Filter (metadata) → **Query Expansion** (đồng nghĩa) → **Decomposition** (tách multi-hop) → **Step-Back** (khái quát hóa) → **Multi-Query** (nhiều biến thể) → **HyDE** (giả lập answer) → Search.

Agentic RAG

Cấp độ cao nhất: hệ thống không chỉ tìm kiếm thụ động mà chủ động suy luận để giải quyết vấn đề

07

Self-Query

Agent tự **tách câu hỏi phức tạp** thành các câu hỏi nhỏ để tìm kiếm nhiều lần, rồi tổng hợp kết quả.

Corrective RAG (C-RAG)

Agent tự **đánh giá chất lượng** tài liệu tìm được. Nếu tài liệu rác → tự động kích hoạt **tìm kiếm Web** hoặc báo lỗi.

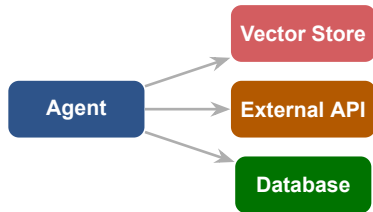
Adaptive RAG

Agent tự **chọn chiến thuật**: câu hỏi dễ → trả lời ngay; câu hỏi khó → kích hoạt tìm kiếm chuyên sâu.

Lưu ý: Agentic RAG = RAG + reasoning loop. Agent không chỉ search 1 lần mà **lặp lại, tự sửa, tự quyết định** chiến thuật.

Agent có khả năng **gọi API bên ngoài** thay vì chỉ đọc dữ liệu tĩnh trong Vector Database:

- Gọi API eDocman để kiểm tra **trạng thái văn bản hiện tại**
- Gọi API JIRA để lấy **ticket mới nhất**
- Gọi calculator, code interpreter, database query
- Kết hợp kết quả từ nhiều tool → tổng hợp câu trả lời



Agentic RAG mở rộng “R” từ chỉ vector search sang **bất kỳ nguồn dữ liệu nào** agent có thể gọi.

RAG Evaluation

Không có evaluation thì không biết hệ thống đang tốt lên hay chỉ đang thay đổi cách viết câu trả lời

08

Kiểm thử thủ công

- Hỏi vài câu, thấy “ổn” → deploy
- Không phát hiện edge case
- Không biết regression khi đổi config
- Không có bằng chứng cho stakeholder

Evaluation có hệ thống

- Bộ test cố định, chạy lại được
- Phát hiện lỗi do Retriever hay Generator
- So sánh trước/sau khi thay đổi
- Báo cáo bằng số liệu cho PM/sếp

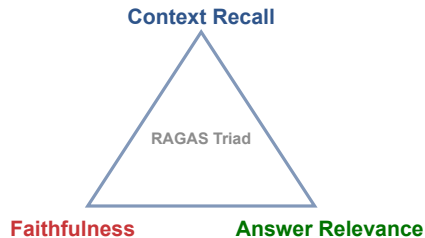
Lưu ý: “Cảm giác ổn” không phải metric. RAG cần **test set + scorecard + so sánh A/B** để biết thực sự đang tốt lên.

Không thể chấm điểm RAG bằng **1 con số duy nhất**.
Phải tách bạch:

- Lỗi do **Retriever** (tìm sai tài liệu)?
- Lỗi do **Generator** (nói bậy dù có tài liệu đúng)?
- Lỗi do **Augmentation** (đóng gói context sai)?

Khung RAGAS chia thành **3 trục cốt lõi** (The Triad):

- 1. Context Recall** — Retriever có tìm đủ không?
- 2. Faithfulness** — Generator có bám chứng cứ không?
- 3. Answer Relevance** — Câu trả lời có đúng ý không?



Context Recall — Retriever có mang về **đủ chứng cứ cần thiết** để trả lời câu hỏi không?

Đo gì:

- Tỷ lệ thông tin cần thiết (ground truth) được tìm thấy trong retrieved context
- Recall thấp = retriever bỏ sót tài liệu quan trọng

Thấp → sửa ở đâu:

- Retrieval strategy (hybrid, rerank)
- Chunking (chunk quá to, thiếu metadata)
- Query transformation (expansion, decomposition)
- Embedding model (đổi model phù hợp domain)

Faithfulness — Câu trả lời của AI có **hoàn toàn bám sát chứng cứ** đã truy xuất không? Hay tự chế thêm?

Đo gì:

- Mỗi claim trong answer có evidence trong context không?
- Claim không có evidence = hallucination
- Tách answer thành các statements, kiểm tra từng cái

Thấp → sửa ở đâu:

- Generation prompt (thêm strict grounding)
- Self-correction loop
- Giảm temperature = 0
- Đặt rule quan trọng ở cuối prompt

Answer Relevance — Câu trả lời có **đúng ý người dùng** hỏi không? Có lạc đề hay thiếu ý không?

Đo gì:

- Answer có trả lời đúng câu hỏi hay lạc đề?
- Có đủ thông tin hay quá ngắn / quá dài?
- Sinh câu hỏi ngược từ answer, so với câu hỏi gốc

Thấp → sửa ở đâu:

- Prompt instruction (format, scope)
- Augmentation (context ordering, compression)
- Model selection (model lớn hơn cho câu hỏi phức tạp)

Context Recall	Faithfulness	Answer Relevance	Chẩn đoán
Cao	Cao	Cao	Hệ thống hoạt động tốt
Thấp	Cao	Thấp	Sửa Retrieval (search sai)
Cao	Thấp	Cao	Sửa Generation (model bịa thêm)
Thấp	Thấp	Thấp	Sửa Indexing (dữ liệu gốc có vấn đề)
Cao	Cao	Thấp	Sửa Augmentation (context đúng nhưng đóng gói sai)

Recall Cao + Faithfulness Thấp

Tìm đúng tài liệu, nhưng model bị ảo giác hoặc prompt viết dở. → **Sửa Generation**.

Recall Thấp + Faithfulness Cao

Hệ thống ngoan ngoãn nói “Tôi không biết” vì không tìm thấy tài liệu. → **Sửa Retrieval**.

V1: Dense Only

- Context Recall: **60%**
- Bỏ lỡ mã lỗi, tên riêng, số ticket
- Faithfulness trung bình vì context thiếu

V2: Hybrid (BM25 + Vector)

- Context Recall: **90%** (↑ 30%)
- Bắt được exact term nhờ BM25
- Faithfulness tăng theo vì context đầy đủ hơn

Chỉ cần thêm BM25 vào pipeline, Context Recall tăng vọt. Đây là thay đổi **ROI cao nhất** cho hầu hết RAG system.

Ví dụ: Thêm Cross-encoder Reranker

- Answer Relevance tăng **+5%**
- Latency tăng từ 1s → **4s**
- Chi phí server tăng **gấp đôi**

Bài toán của kỹ sư trưởng

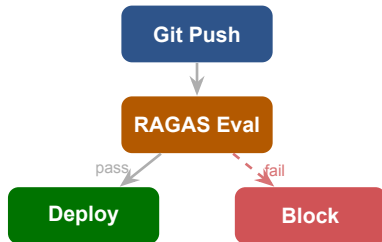
5% độ chính xác đó có đáng giá với:

- Trải nghiệm chậm chạp của người dùng?
- \$500/tháng chi phí thêm?
- +3s latency mỗi request?

Lưu ý: Không phải kỹ thuật nào cũng đáng thêm. Mỗi cải tiến cần cân nhắc **quality gain vs. latency vs. cost**.

Code RAG $\neq$ Code Web

- Khi deploy RAG, bạn test **hành vi của AI**
- Tích hợp RAGAS vào **GitHub Actions**
- Mỗi lần đổi config $\rightarrow$ chạy eval tự động
- Faithfulness $< 80\%$ $\rightarrow$ **block deploy**



Không có eval tự động = không biết regression. Treat RAG eval như **unit test cho AI**

Multi-Agent, MCP & A2A

“Một agent rất giỏi vẫn có giới hạn. Khi bài toán lớn hơn một vòng RAG, ta cần supervisor, worker, và giao thức kết nối. ”

- Chuẩn bị 1 use case mà single-agent đang bắt đầu quá tải
- Nghĩ trước: nếu tách 2–3 worker thì nên chia theo tool, domain, hay bước xử lý?
- Xem lại artifact Day 08 để xác định phần nào có thể thành 1 worker độc lập

1. Lewis et al. (2020), *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*.
2. OpenAI Docs, *Retrieval Guide* và *File Search Guide*.
3. LangChain, *RAG from Scratch* notebooks.
4. LlamaIndex Docs, *Starter Example*.
5. RAGAS Docs, *Evaluation metrics for RAG systems*.
6. Cohere Docs, *Rerank overview*.
7. Liu et al. (2023), *Lost in the Middle: How Language Models Use Long Contexts*.
8. Yan et al. (2024), *Corrective Retrieval Augmented Generation (C-RAG)*.
9. Es et al. (2023), *RAGAS: Automated Evaluation of Retrieval Augmented Generation*.

Hỏi & Đáp

Bạn đang thiếu model mạnh hơn, hay đang thiếu một pipeline retrieval, augmentation, và evaluation đủ kỷ luật?